

La solution : planifier avec BuiltInPlanner

Activer le raisonnement en plusieurs étapes

Référence : [Documentation ADK — Planification](#)

D'après la documentation ADK, utilisez `BuiltInPlanner` pour exploiter la fonctionnalité de réflexion de Gemini :

"planner (facultatif) : attribuez une instance `BasePlanner` pour activer le raisonnement et la planification en plusieurs étapes avant l'exécution.
`BuiltInPlanner` : exploite les capacités de planification intégrées du modèle (par exemple, la fonctionnalité de réflexion de Gemini)."

```
from google.adk.agents import LlmAgent
from google.adk.planners import BuiltInPlanner
from google.genai import types

planning_agent = LlmAgent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(
            include_thoughts=True,    # Affiche le raisonnement dans la réponse
            thinking_budget=1 024     # Jetons alloués au raisonnement
        )
    ),
    instruction="Résous des problèmes complexes de manière systématique"
)
```

Remarque : Cette fonctionnalité utilise les capacités de [raisonnement de Gemini](#).

Voir la planification en action

Comparons le même problème avec et sans planification pour comprendre ce qu'elle apporte réellement :

Exemple de prompt : "Dois-je acheter ou louer une voiture pour mon trajet quotidien de 80 km ?"

Sans planification

```
basic_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="Aide les utilisateurs à prendre des décisions"
)
```

Réponse typique sans planification :

La location propose des paiements mensuels plus faibles et vous permet d'avoir une voiture neuve à intervalles de quelques années.
En achetant une voiture, elle vous appartient et vous pouvez parcourir un nombre illimité de kilomètres. Pour un trajet quotidien de 80 km, acheter est probablement préférable, car les contrats de location imposent des limites de kilométrage.

Éléments manquants :

- Analyse des implications financières
 - Prise en compte du coût total sur la durée
 - Quantification du problème de kilométrage (80 km/jour = environ 29 000 km/an)
 - Comparaison de différentes mises en situation
-

Avec BuiltInPlanner

```
planning_agent = LlmAgent(  
    model="gemini-2.5-flash",  
    planner=BuiltInPlanner(  
        thinking_config=types.ThinkingConfig(  
            include_thoughts=True,      # Affiche le raisonnement  
            thinking_budget=1 024      # Alloue des jetons au raisonnement  
        )  
    ),  
    instruction="Aide les utilisateurs à prendre des décisions"  
)
```

Réponse avec planification (include_thoughts=True affiche le raisonnement) :

[Raisonnement de l'agent - visible, car include_thoughts=True]
Analysons cela de manière systématique :
- 80 km/jour × 5 jours/semaine × 52 semaines = ~20 800 km/an
- La plupart des contrats de location imposent une limite de 16 000-19 000 km/an - ce qui frôle la limite pour ce trajet
- Il est nécessaire de comparer : le coût total, la flexibilité, les frais de dépassement de kilométrage
- À prendre en compte : la durée pendant laquelle la voiture sera conservée

Facteurs clés :

1. Limites de kilométrage des contrats de location : 19 000 km pour un contrat standard. Il faudrait opter pour un forfait à 24 000 km (paiement plus élevé)
2. Achat : paiement mensuel plus élevé mais constitution de capital
3. Les kilomètres parcourus impactent la valeur de revente de votre véhicule

Je vais créer différentes mises en situation...

[Réponse finale]

Voici une comparaison pour votre commute trajet quotidien de 80 km par jour (environ 21 000 km par an) :

****Location :****

- Coût mensuel : 300 € (avec le forfait de 24 000 km/an - vous dépasseriez 19 000 km du forfait standard)
- Total sur 3 ans : environ 10 800 € en plusieurs paiements
- Résultat final : restitution du véhicule, aucun capital accumulé
- Risque : frais de dépassement si vous franchissez la limite de kilométrage (0,22 €/km ce qui peut vite représenter une somme importante)

****Achat :****

- Coût mensuel : 387 € (prêt typique sur cinq ans)
- Total sur 3 ans : environ 14 000 € en plusieurs paiements
- Résultat final : posséder un véhicule d'une valeur d'environ 13 000 € (en tenant compte de la forte dépréciation due au kilométrage élevé)
- Coût net : environ 1 000 € en tenant compte du capital accumulé

****Recommandation pour votre situation :****

Mieux vaut acheter que louer. Avec un kilométrage élevé (environ 21 000 km/an), la location devient risquée et coûteuse :

1. Il vous faudrait un forfait de location pour kilométrage plus élevé (plus coûteux).
2. Tout trajet imprévu risque d'entraîner des frais coûteux de dépassement importants.
3. Sur 3 ans, acheter ne coûte qu'environ 1 000 € de plus en tenant compte du capital accumulé.
4. L'achat vous offre une liberté totale, sans restriction de kilométrage.

Exception : si vous privilégiez des paiements mensuels plus bas plutôt que le coût total sur le long terme, un forfait de location de 24 000 km peut vous convenir, à condition de prévoir un budget pour les éventuels frais de dépassement.

Bénéfices de la planification :

- **Décomposition du problème** : détail du calcul du kilométrage et des facteurs de coût
- **Analyse multifactorielle** : comparaison du coût total, du capital accumulé, des risques et de la flexibilité
- **Calculs précis** : calcul de valeurs réelles au lieu de généralités
- **Comparaison de mises en situation** : examen des deux options dans un contexte spécifique
- **Recommandation nuancée** : conseil principal + exception

Point essentiel concernant les instructions :

L'instruction est restée simple : "aide les utilisateurs à prendre des décisions." Il n'est pas nécessaire de rédiger des instructions complexes de type "décompose le problème en plusieurs phases" ou "tiens compte des compromis et des contraintes". La capacité de planification permet ce raisonnement structuré automatiquement : **thinking_budget** détermine la profondeur de réflexion du modèle, et **include_thoughts** vous permet de voir (et de déboguer) ce processus de raisonnement.

Concepts fondamentaux

1. Comprendre le paramètre **planner**

Référence : [Documentation ADK — Paramètre "planner"](#)

Le paramètre **planner** (au singulier) active la planification :

```
from google.adk.planners import BuiltInPlanner

agent = LlmAgent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(...) # Au singulier : planner=, et non planners=
)
```

Points essentiels :

- Fonctionnalité disponible uniquement en Python actuellement
 - Exploite les capacités de réflexion intégrées du modèle
 - Idéal pour les problèmes complexes nécessitant plusieurs étapes
-

2. Configurer ThinkingConfig

Référence : [Documentation ADK – Paramètres ThinkingConfig](#)

D'après la documentation :

"Ici, le paramètre **thinking_budget** fournit au modèle des indications sur le nombre de jetons de réflexion qu'il peut utiliser pour générer une réponse. Le paramètre **include_thoughts** détermine si le modèle doit inclure ses réflexions brutes et son processus de raisonnement dans la réponse."

```
from google.genai import types

# Configuration avec raisonnement visible
transparent_config = types.ThinkingConfig(
    include_thoughts=True,    # Affiche le raisonnement de l'agent
    thinking_budget=1 024    # Alloue 1 024 jetons au raisonnement
)

# Configuration avec raisonnement masqué
opaque_config = types.ThinkingConfig(
    include_thoughts=False,  # Masque le raisonnement
    thinking_budget=512      # Utilise moins de jetons pour le raisonnement
)
```

Explication des paramètres :

- **include_thoughts** : si la valeur est définie sur "true", la réponse inclut le processus de raisonnement de l'agent.
- **thinking_budget** : nombre de jetons alloués au raisonnement (non inclus dans la

réponse finale)

3. Choisir parmi les planificateurs

Référence : [Documentation ADK — Types de planificateurs](#)

L'ADK propose deux types de planificateurs, chacun adapté à des situations différentes :

BuiltInPlanner (recommandé pour les modèles Gemini)

Exploite les capacités de raisonnement natives du modèle. Extrait de la documentation ADK :

BuiltInPlanner : exploite les capacités de planification intégrées du modèle (par exemple, la fonctionnalité de raisonnement de Gemini)."

Recommandé pour :

- **Modèles Gemini** : 2.5 Flash, 2.5 Pro, etc. dotés de capacités de raisonnement intégrées
- **Raisonnement transparent** : affichez le processus de raisonnement avec `include_thoughts=True`.
- **Contrôle flexible** : ajustez la profondeur du raisonnement avec `thinking_budget`.

Configuration :

```
from google.adk.planners import BuiltInPlanner
from google.genai import types

agent = LlmAgent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(
            include_thoughts=True,    # Affiche le raisonnement (idéal pour le
débogage)
            thinking_budget=1 024    # jetons alloués au raisonnement (généralement
512 à 2 048)
        )
    )
)
```

Paramètres clés :

- **thinking_budget** : contrôle la profondeur du raisonnement du modèle. Les valeurs les plus élevées (par exemple, 2048) permettent une analyse plus approfondie des problèmes complexes. Les valeurs les plus basses (par exemple, 512) offrent une exécution plus rapide pour des tâches simples.
- **include_thoughts** : lorsque la valeur est définie sur "True", le processus de raisonnement du modèle s'affiche dans la réponse. Indispensable pour le débogage et pour comprendre comment le cheminement de l'agent.

PlanReActPlanner (pour les modèles autres que Gemini)

Référence : [Documentation ADK — PlanReActPlanner](#)

Extrait de la documentation ADK :

"PlanReActPlanner" : ce planificateur demande au modèle de suivre une structure spécifique dans son résultat, à savoir créer un plan, puis exécuter des actions (comme l'appel d'outils) tout en fournissant son raisonnement pour chaque étape. *Il est particulièrement utile pour les modèles qui ne disposent pas d'une fonctionnalité de "raisonnement" intégrée.*

Recommandé pour :

- **Modèles autres que Gemini** : par exemple, d'autres LLM sans capacités de raisonnement natives
- **Format de sortie structurée** : impose la structure PLANNING → ACTION → REASONING → FINAL_ANSWER
- **Workflows utilisant de nombreux outils** : les phases d'action/de raisonnement explicites fonctionnent bien avec les appels d'outils.
- **Application d'une approche systématique** : lorsque vous avez besoin d'une structure de sortie garantie

Configuration :

```
from google.adk.planners import PlanReActPlanner

agent = LlmAgent(
    model="your-model", # N'importe quel modèle, en particulier les modèles autres
    que Gemini
    planner=PlanReActPlanner()
)
```

Exemple de structure de sortie :

Selon la documentation ADK, PlanReActPlanner produit une sortie structurée comme suit :

[utilisateur] : Aide-moi à organiser un événement d'équipe hors site

[agent] : /*PLANNING*/

1. Déterminer la taille de l'équipe et les contraintes budgétaires
2. Suggérer des options de lieux en fonction de la taille de l'équipe
3. Proposer un programme d'activités
4. Fournir une checklist logistique

/*ACTION*/

[Les éventuels appels d'outils apparaîtraient ici]

/*REASONING*/

Un événement d'équipe hors site réussi nécessite à la fois des activités structurées et du temps libre.

Combiner des activités de team-building avec des moments de socialisation génère un

meilleur
engagement que des formats exclusivement professionnels ou sociaux.

/*FINAL_ANSWER*/

Voici une suggestion de programme pour votre événement d'équipe hors site :

Lieu : tenir compte [des recommandations spécifiques]

Programme : ateliers le matin, activités d'équipe l'après-midi, moments de convivialité le soir

Budget : 130-170 € par personne pour le lieu + les activités

...

Quel planificateur devriez-vous utiliser ?

- **Utilisez BuiltInPlanner** avec les modèles Gemini (2.5 Flash, 2.5 Pro, 2.0 Flash).
 - Processus de raisonnement plus naturel
 - Profondeur de raisonnement configurable avec `thinking_budget`
 - Affichage facultatif du raisonnement avec `include_thoughts`
- **Utilisez PlanReActPlanner** dans les cas suivants :
 - Lorsque vous utilisez des modèles autres que Gemini dépourvus de capacités de raisonnement intégrées
 - Lorsque vous avez besoin d'une structure de sortie stricte (PLANNING/ACTION/REASONING/FINAL_ANSWER)
 - Lorsque vous créez des agents qui utilisent beaucoup d'outils et pour lesquels les phases d'action explicites sont utiles

Dans ce cours, nous nous concentrons sur **BuiltInPlanner**, car nous utilisons des modèles Gemini. PlanReActPlanner est abordé dans les cours avancés sur les systèmes multi-agents.

4. Quand utiliser la planification

La planification est adaptée dans les cas suivants :

-  Résolution de problèmes en plusieurs étapes
-  Tâches nécessitant une analyse des compromis
-  Prise de décisions complexes
-  Débogage du raisonnement de l'agent

La planification n'est pas adaptée dans les cas suivants :

-  Questions simples et directes
-  Recherches rapides d'informations factuelles
-  Tâches nécessitant une seule étape
-  Réponses urgentes

Utilisation optimale de la planification

```

strategic_agent = LlmAgent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(
            include_thoughts=True,
            thinking_budget=2 048 # Budget important pour une analyse complexe
        )
    ),
    instruction="Analyse les stratégies d'entreprise et propose des recommandations"
)

# Utilisation superflue de la planification
simple_agent = LlmAgent(
    model="gemini-2.5-flash",
    # Pas de planificateur - tâche simple
    instruction="Offre un accueil chaleureux aux utilisateurs"
)

```

5. Débogage avec affichage du raisonnement

Activez `include_thoughts=True` pour afficher le raisonnement de l'agent :

```

debug_agent = LlmAgent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(
            include_thoughts=True, # Affiche le raisonnement
            thinking_budget=1 024
        )
    ),
    instruction="Aide les utilisateurs à déboguer les problèmes de manière systématique"
)

# Contenu de la réponse :
# 1. Processus de réflexion de l'agent
# 2. Décomposition du problème
# 3. Réponse finale

```



Exemple pratique

Créons un agent de résolution de problèmes qui utilise la planification pour répondre à des questions complexes.

Étape 1 : Créer le projet

```

adk create problem_solver
cd problem_solver

```

Étape 2 : Écrire l'agent avec planification intégrée

Remplacez le contenu de `agent.py` par le code suivant :

```
"""
Agent de résolution de problèmes avec des capacités de planification intégrées.
Exemple d'utilisation de BuiltInPlanner d'ADK avec ThinkingConfig.
"""

from google.adk.agents import LlmAgent
from google.adk.planners import BuiltInPlanner
from google.genai import types

# Agent avec planification intégrée pour la résolution de problèmes complexes
root_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="strategic_problem_solver",
    description="Résout des problèmes complexes en utilisant un raisonnement et une planification en plusieurs étapes",
    instruction="""Tu es un spécialiste de la résolution de problèmes stratégiques.

Ton approche des problèmes complexes :
1. **Comprendre** : décomposer le problème en plusieurs parties
2. **Analyser** : examiner plusieurs approches et compromis
3. **Planifier** : élaborer une stratégie de solution étape par étape
4. **Exécuter** : fournir des recommandations claires et concrètes

Pour les problèmes complexes :
- Examiner les différentes implications et les cas particuliers
- Prendre en compte les conséquences à court et long terme
- Identifier les risques potentiels et les stratégies d'atténuation
- Fournir le raisonnement derrière tes recommandations

Sois rigoureux, analytique et méthodique dans ton approche.""",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(
            include_thoughts=True, # Affiche le raisonnement
            thinking_budget=2048 # Budget important pour un raisonnement complexe
        )
    )
)
```

Étape 3 : Exécuter et tester

adk web

Test 1 : Problème complexe en plusieurs étapes

Essayez ce prompt :

Comment préparer ma transition professionnelle de l'ingénierie logicielle vers le product management au cours des 12 mois à venir ?

Comportement attendu avec la planification :

- L'agent montre son processus de raisonnement
- Il divise la transition en plusieurs phases et identifie les compétences requises
- Il examine plusieurs stratégies de préparation
- Il analyse les compétences à développer par ordre de priorité
- Il fournit une feuille de route structurée et progressive

Résultat :

[Processus de raisonnement - visible, car include_thoughts=True]

- Analyse des écarts de compétences : compétences en ingénierie (acquises) et compétences en product management (à développer)
- Compétences clés en PM : stratégie produit, recherche utilisateur, gestion des personnes concernées, élaboration de feuille de route
- Compétences transférables de l'ingénierie logicielle : connaissances techniques, analyse des données, exécution de projets
- Échéance : 12 mois, ce qui permet une transition progressive tout en restant employé
- Stratégie : développer des compétences - acquérir de l'expérience en PM - réseautage
- transition

[Réponse finale]

Voici une feuille de route sur 12 mois pour réussir à passer de l'ingénierie logicielle au product management:

Mois 1-3 : acquérir les bases du PM

- Lire des ouvrages clés en PM (Inspired, Lean Product Playbook)
- Suivre une formation en ligne de PM (Reforge, Product School)
- Commencer à rédiger des spécifications produit pour les fonctionnalités de votre équipe actuelle
- Observer votre PM actuel lors des réunions

Mois 4-6 : développer des compétences clés en PM

- Piloter intégralement le développement d'une petite fonctionnalité (des spécifications au lancement)
- Conduire des entretiens utilisateurs pour le produit de votre équipe
- S'exercer à la gestion des personnes concernées dans votre poste actuel
- Constituer un portfolio de réalisations produit (spécifications, feuilles de route, métriques analyse)

Mois 7-9 : acquérir de l'expérience en PM et renforcer votre visibilité

- Participer bénévolement à des projets liés au product management (planification de feuilles de route, recherche utilisateur)
- Lancer un projet parallèle en PM pour mettre en avant vos compétences
- Réseauter avec des responsables produit (entretiens d'information, rencontres professionnelles)
- Se faire accompagner par un PM en exercice

Mois 10 à 12 : recherche active d'emploi

- Modifier votre CV pour mettre en avant vos succès liés au PM
- Postuler à des programmes APM ou à des postes de PM dans des petites entreprises
- Exploiter les opportunités en interne si disponibles
- Se préparer à assumer un rôle de PM en tant que contributeur individuel (et non

senior) dans un premier temps

Cette approche progressive vous permet d'acquérir des compétences tout en continuant à travailler et permet de réduire les risques.

Test 2 : Question simple et directe (remarquez la différence)

Essayez ce prompt :

Quelle est la capitale de la France ?

Comportement attendu :

- Même avec la planification activée, les questions simples reçoivent des réponses directes
- Budget de raisonnement minime pour les questions simples
- L'agent adapte la complexité en fonction de la tâche

Test 3 : Analyse des compromis

Essayez ce prompt :

Devrais-je louer ou acheter une maison à San Francisco si je prévois d'y rester sur une période de 3 à 5 ans ?

Comportement attendu avec la planification :

- Examine les deux options de façon systématique
- Prend en compte les implications financières (apport, mensualités, valeur résiduelle, coût d'opportunité)
- Analyse des facteurs spécifiques au marché (prix de l'immobilier à San Francisco, encadrement des loyers, plus-value)
- Évalue l'impact sur un horizon de trois à cinq ans
- Fournit des recommandations contextuelles accompagnées de mises en situation

Points à noter :

- Le processus de raisonnement est visible lorsque `include_thoughts=True`.
- Les problèmes complexes sont analysés de manière structurée grâce à la planification.
- Les questions simples restent efficaces même lorsque le planificateur est activé.
- Le budget limite la profondeur de la réflexion (2 048 jetons = analyse approfondie).

Points à retenir

- Le **paramètre planner** (au singulier) permet un raisonnement en plusieurs étapes.
- **BuiltInPlanner** exploite les capacités de réflexion intégrées de Gemini.
- **ThinkingConfig** contrôle le budget et la visibilité du raisonnement
- `include_thoughts=True` affiche le raisonnement (idéal pour le débogage).
- **thinking_budget** alloue des jetons pour le raisonnement (distincts de la réponse).

- **La planification est utile pour les tâches complexes**, mais pas pour les questions simples.
- **L'importation est à effectuer depuis `google.adk.planners` et `google.genai.types`**